

AI技术应用场景匹配报告

项目名称：家具购物网站

学号：2023141010307

姓名：赵一鸣

日期：2026年6月

一、引言

随着大语言模型（LLM）、生成式AI、智能体（AI Agent）等技术的爆发式发展，软件工程正经历从传统流程驱动模式向人机协同智能范式的深刻转型。2024年至2026年间，AI技术在软件工程各阶段的应用已从概念验证走向工程落地，涌现出大量成熟的应用模式和工具链。

本报告基于家具购物网站课程项目的软件过程定义，系统调研AI技术在软件生命周期各阶段的最新应用成果，识别出项目中可借助AI技术显著提升效率的关键环节，为后续的过程改进方案设计提供依据。

二、AI在软件工程各阶段的应用现状

2.1 需求分析阶段

（1）AI辅助需求捕获

基于LLM的对话式需求引导工具（如ChatGPT、Claude）能够通过结构化提问帮助分析师系统化地挖掘用户需求。2024年后，此类工具已支持多轮上下文感知对话，可自动识别需求中的歧义、遗漏和不一致性。

（2）需求文档自动生成

AI工具能够根据自然语言对话记录自动生成结构化的需求规格说明书（SRS），包括用例描述、功能需求列表、非功能需求分类等。研究表明，使用AI辅助生成需求文档可减少约40%-60%的文档编写时间[1]。

（3）需求一致性验证

基于NLP的需求验证工具可自动检测需求之间的冲突、重复和遗漏，显著提升需求评审的效率。2025年，基于知识图谱的需求一致性检查技术已进入实用阶段[2]。

2.2 系统设计阶段

(1) AI驱动的架构设计

AI工具可根据需求文档自动推荐技术架构方案，包括技术栈选型、系统分层设计、组件划分等。GitHub Copilot Chat等工具支持在IDE中直接讨论架构方案，提供设计模式的实时推荐。

(2) 自动生成UML模型

基于自然语言描述，AI可自动生成类图、时序图、用例图等UML模型。Mermaid代码形式的图表支持版本控制和协作编辑，极大地降低了设计文档的维护成本。

(3) 设计模式推荐

AI能识别代码中的反模式（Anti-Pattern），并针对特定场景推荐合适的设计模式，提升代码质量和可维护性。

2.3 编码实现阶段

(1) AI代码生成

GitHub Copilot、Cursor、Amazon CodeWhisperer等工具已实现行级、函数级甚至文件级的代码自动补全。2025年的研究表明，使用AI代码生成工具可将编码效率提升30%-55%[3]。

(2) 智能代码审查

AI代码审查工具（如CodeRabbit、Amazon CodeGuru）可自动发现安全漏洞、性能问题、代码异味和最佳实践偏离，提供上下文感知的改进建议。

(3) 自动代码重构

AI工具能识别代码中的技术债务，建议并自动执行安全的重构操作，包括变量重命名、函数提取、设计模式应用等。

2.4 测试验证阶段

(1) AI生成测试用例

基于需求和代码上下文，AI可自动生成单元测试、集成测试用例，覆盖正常路径、边界条件和异常场景。根据Gtest全球软件测试技术峰会2025年报告，AI生成测试用例的覆盖率达到人工编写的85%-95%，而时间消耗仅为人工的20%-30%[4]。

(2) 智能自愈测试脚本

当UI或API发生变更时，AI测试工具（如Testim、Mabl）能自动识别变更并更新测试脚本，减少测试维护工作量。

(3) 预测性缺陷检测

基于历史缺陷数据训练的ML模型能够预测高缺陷风险模块，指导测试资源的优化分配。

2.5 运维监控阶段

(1) AI异常检测

基于机器学习的异常检测系统能自动识别系统指标中的异常模式，在问题影响用户之前发出预警。

(2) 根因自动分析

AI驱动的根本原因分析（RCA）工具可将故障定位时间从小时级缩短到分钟级，通过关联分析多维度监控数据自动推断故障原因。

三、项目关键环节识别与AI匹配分析

结合家具购物网站项目的软件过程定义和项目特点（小型课程项目、4人团队、10天开发周期），以下识别出5个AI技术可显著提升效率的关键环节：

3.1 关键环节一：需求分析与文档编写

现状问题：当前过程定义中，需求分析完全依赖人工完成，包括需求调研、需求规格说明书编写、需求评审等。对于小型课程项目，虽然需求相对简单，但需求文档的质量直接影响后续所有环节。

AI匹配方案：使用LLM（如Claude或ChatGPT）辅助需求引导和文档生成。通过对话式交互，快速梳理用户故事（User Stories），自动生成结构化的需求规格说明书。

预期提升：需求文档编写时间减少50%，需求完整性提升20%。

3.2 关键环节二：系统设计与架构决策

现状问题：家具购物网站需要技术选型（前端框架、后端语言、数据库选择）、系统架构设计、数据库设计、接口设计等。对于经验不足的学生团队，设计决策往往缺乏充分的对比分析。

AI匹配方案：使用AI工具辅助技术选型分析和架构设计。利用AI生成Mermaid格式的UML模型（类图、时序图、ER图），自动生成API接口文档框架。

预期提升：设计阶段时间减少30%，设计文档完整性提升40%。

3.3 关键环节三：前端与后端编码

现状问题：家具购物网站涉及用户注册登录、商品浏览与搜索、购物车管理、订单处理等模块的编码工作，编码量大且容易出错。

AI匹配方案：使用GitHub Copilot或Cursor进行AI辅助编码，包括代码自动补全、样板代码生成、CRUD模块快速生成。

预期提升：编码效率提升35%-50%，代码缺陷率降低25%。

3.4 关键环节四：测试用例设计与执行

现状问题：当前过程定义中包含单元测试、集成测试、系统测试，但课程项目往往测试覆盖不足，手工编写测试用例耗时且容易遗漏边界情况。

AI匹配方案：使用AI工具（如GitHub Copilot或Claude）自动生成测试用例，覆盖单元测试和集成测试。AI辅助生成测试数据，包含边界值和异常场景。

预期提升：测试用例编写时间减少60%，测试覆盖率提升30%。

3.5 关键环节五：代码评审与质量保证

现状问题：当前过程定义中的代码评审为简化版内部评审，但人工代码评审耗时且评审深度受限
于评审者经验。

AI匹配方案：使用AI代码审查工具（如CodeRabbit或GitHub Copilot Code Review）进行预审，自动标记安全漏洞、代码异味和最佳实践偏离，人工在此基础上做终审。

预期提升：代码评审时间减少40%，缺陷检出率提升20%。

四、场景匹配总结

关键环节	原始过程活动	AI技术/工具	匹配度	预期效率提升
需求分析与文档	需求分析、编写SRS	LLM对话引导+文档生成	高	文档编写时间-50%
系统设计	架构设计、详细设计	AI架构推荐+UML生成	中高	设计时间-30%
编码实现	前端/后端开发	GitHub Copilot/Cursor	高	编码效率+35-50%
测试用例	单元/集成/系统测试	AI测试生成+自愈脚本	高	测试编写时间-60%
代码评审	代码评审	AI代码审查工具	中高	评审时间-40%

匹配原则：

- 1. **高匹配度环节**优先进行AI工具原型验证，确保可落地性
- 2. **可量化效率提升**作为后续过程改进方案的预期目标
- 3. AI技术作为**辅助工具**而非完全替代人工，保留关键决策权

4. 考虑课程项目的时间约束和团队能力，选择**成熟度高、学习成本低**的AI工具